

5

Supervised Fine-Tuning

Supervised Fine-Tuning (SFT) is a crucial step in preparing LLMs for real-world applications. Following the initial pre-training phase, where an LLM learns to predict the next token in a sequence, SFT refines the model's capabilities using carefully curated pairs of instructions and corresponding answers. This process serves two primary purposes: it teaches the model to understand and follow a specific chat format, effectively transforming it into a conversational agent, and it allows the model to adapt its broad knowledge base to excel in targeted tasks or specialized domains.

The importance of SFT lies in its ability to bridge the gap between a model's general language understanding and its practical utility. By exposing the model to examples of desired input-output patterns, SFT shapes the LLM's behavior to align with specific goals, whether they involve task completion (such as summarization or translation) or domain expertise (like medical or legal knowledge). This tailored approach not only enhances the model's performance in intended areas but also improves its ability to follow instructions and generate more relevant and coherent responses.

In this chapter, we will cover the following topics:

- Creating a high-quality instruction dataset
- SFT techniques
- Implementing fine-tuning in practice

By the end of this chapter, you will be able to create your own instruction datasets and efficiently fine-tune LLMs on them.



All the code examples from this chapter can be found on GitHub at <https://github.com/PacktPublishing/LLM-Engineering>.

Creating an instruction dataset

In most use cases, creating an instruction dataset is the most difficult part of the fine-tuning process. This is due to multiple factors. Most use cases can be connected to raw text, but it is rare to find natural pairs of instructions and answers. This raw text needs to be transformed into a format that includes both instructions and answers. Moreover, the quality of the data is also crucial. Because of this, a lot of time is invested in manually checking and verifying individual samples. This careful review helps ensure that the dataset is accurate and useful for training the model.

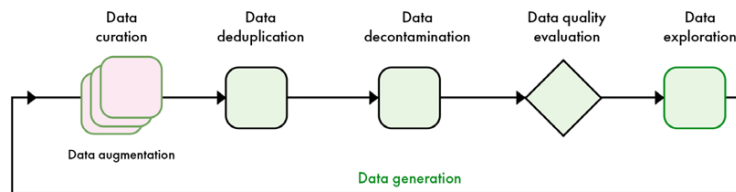


Figure 5.1 – Overview of the post-training data pipeline covered in this chapter

In this section, we will introduce a general framework to create your own instruction datasets, regardless of the final use case. We will then leverage the scraped data from *Chapter 3* and transform it into an instruction dataset. The different stages in our data generation pipeline are summarized in *Figure 5.1*.

General framework

Instruction datasets are defined as pairs of instructions and answers. The instructions are the inputs of the model, used as context during fine-tuning. The answers are the expected outputs of the model. During fine-tuning, you can choose to train the model on the instructions and answers, or on answers only. Pairs of instructions and answers follow a certain template. Some instruction templates, such as Alpaca, introduce additional fields like `inputs` and `system`. Both of them can be considered sub-fields of the `instruction` field. In this case, “inputs” contain the data the model needs to complete the instruction, and “system” is a meta-prompt to steer the general behavior of the model. Here is an example from the SlimOrca dataset, with “system” and “instruction”:

System

You are a helpful assistant, who always provide explanation. Think like you are answering to a five year old.

Instruction

Concepts: building, shop, town

Write a sentence that includes all these words.

Output

In our little town, there is a shop inside a big building where people go to buy their favorite toys and candies.

Table 5.1 – Example of sample from the Open-Orca/SlimOrca dataset

This example illustrates how the “system” field is used to define specific behaviors for the model, such as being helpful, always providing explanations, and tailoring responses as if speaking to a five-year-old. The “instruction” field provides the necessary data (the concepts) and the task (constructing a sentence). The `output` field shows the expected answer, which, while not the only possible answer, represents a high-quality response.

To build an instruction dataset, we want to curate data that is representative of how the model will be used. Once we have gathered enough sam-

ples, our goal is to filter them to only keep high-quality data. In this context, high-quality data can be described through three main dimensions:

- **Accuracy:** It refers to the factual correctness and relevance of the samples. In the context of instruction datasets, this means ensuring that responses are not only factually accurate but also relevant to their corresponding instructions. High accuracy is essential for training models that can provide reliable and trustworthy information.
- **Diversity:** A high-quality dataset should encompass a wide range of use cases, covering the potential queries and tasks the deployed LLM might encounter. This diversity should span topics, contexts, text lengths, and writing styles. By sampling data in a representative manner, we allow models to develop robust instruction-following capabilities.
- **Complexity:** Trivial or overly simplistic samples do little to improve an LLM's capabilities. Instead, datasets should include complex, multi-step reasoning problems and challenging tasks that push the boundaries of what the model is expected to handle. This complexity helps in developing models capable of tackling complex real-world problems.

In the following sections, we will see techniques to filter and evaluate instruction samples according to these dimensions.

Data quantity

The Hugging Face Hub contains numerous instruction datasets, which can be general-purpose or designed for particular tasks or domains. When working on a new use case, it can be beneficial to look for related open-source datasets to leverage for fine-tuning. This is particularly important if your number of samples is too low (for example, fewer than 1,000), requiring you to augment it with high-quality data.

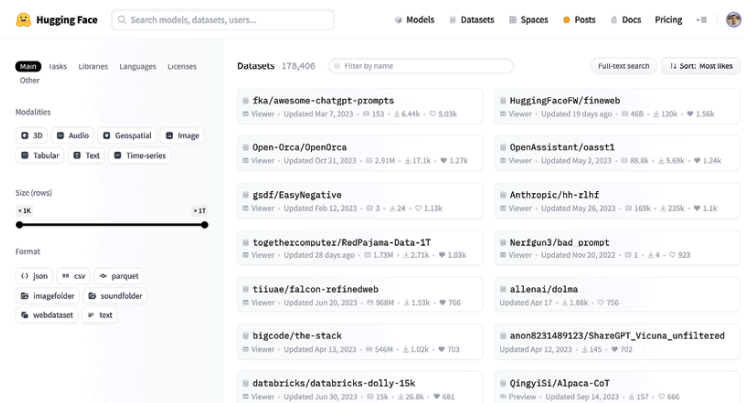


Figure 5.2 – Screenshot of the most-liked datasets on the Hugging Face Hub



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** and a **free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781836200079>.

Calculating an ideal number of samples is a difficult task, as both the quality of the data and the size of the model can have a dramatic impact. For large models (around 70 billion parameters, for example), this number can be as low as 1,000 high-quality samples (see the LIMA paper in the *References* section). This is not true for smaller models (around seven billion parameters, for instance), as they need more samples to simply learn the correct chat template. In any case, the quality of the data is a crucial factor, and a high number of samples is always desirable.

To provide additional numbers, we can look at the fine-tuned models developed by companies and the open-source community. We can distinguish two types of finetunes: general-purpose, aimed to reproduce the capabilities of models like GPT, and task- or domain-specific models, designed to optimize their performance for a particular application.

General-purpose models cover more topics, which requires additional samples. Among companies, we observe a wide range of values. For instance, Yi models from 01-ai rely on less than 10,000 samples. At the opposite range of the spectrum, Meta reported using 10 million samples for Llama 3 through the entire fine-tuning process (including preference alignment). In the open-source community, models like OpenHermes and Dolphin use around one million samples. Based on the quality of these finetunes, we recommend an instruction dataset of at least one million samples to create a good general-purpose instruct model. On the other hand, models fine-tuned for a specific purpose require fewer samples. Here, we differentiate task-specific models from domain-specific ones.

Task-specific and domain-specific models represent two distinct approaches to fine-tuning LLMs. Task-specific models are designed to excel at a particular function, such as translation, summarization, or sentiment analysis. These models benefit from a focused training approach on a single task, allowing for efficient performance even with smaller model sizes (typically less than 8 billion parameters). The data required for task-specific fine-tuning is generally more manageable, ranging from 100 to 100,000 samples. This makes task-specific fine-tuning an attractive option for many applications where resources may be limited.

Domain-specific models, on the other hand, aim to tweak the LLM with specialized knowledge and familiarity with the vocabulary and linguistic patterns of a particular field. These models are valuable in areas such as medicine, law, finance, e-commerce, engineering, and hospitality. The data requirements for domain-specific fine-tuning can vary widely depending on the complexity and breadth of the domain. Some fields, like medicine or law, may require as much data as general-purpose fine-tuning due to their vast technical corpora. Others, such as e-commerce or hospitality, might need fewer samples, more in line with task-specific fine-tuning.

The key factors determining the data needs for domain-specific models are the “size” of the domain (i.e., the extent of its specialized knowledge and vocabulary) and the representation of that domain in the model’s pre-training data. Domains that are well-represented in the original training data may require less fine-tuning, while those that are more specialized or underrepresented may need more extensive datasets. Even with open-source LLMs, many pre-training datasets are closed-source, which

requires making educated guesses to determine their composition (e.g., 30% code or 20% math).

Data curation

When it comes to procuring data for fine-tuning, the approaches differ between task-specific and domain-specific models. For task-specific models, data curation often involves collecting examples of the desired task from existing datasets or creating new ones. This might involve gathering pairs of original and summarized texts for a summarization model or collecting sentences in different languages for a translation model.

Domain-specific data curation can be more challenging. It often requires collaboration with subject matter experts to gather and validate relevant texts, research papers, technical documents, and other domain-specific content. In some cases, it may involve partnering with organizations or institutions that have access to large repositories of specialized information. The quality and relevance of this data is crucial, as it directly impacts the model's ability to understand and generate content in the target domain.

It's worth noting that few-shot prompting has emerged as an alternative strategy to fine-tuning, especially for task-specific applications. This approach leverages the capabilities of large, powerful models by providing a few examples of the desired task within the input prompt. While not a replacement for fine-tuning in all scenarios (e.g., when you want to learn a new domain), few-shot prompting can be an efficient way to adapt models to new tasks without the need for extensive additional training.

In practice, the line between task-specific and domain-specific models can sometimes blur. For instance, a model fine-tuned for medical diagnosis could be considered both task-specific (focused on diagnosis) and domain-specific (specialized in medical knowledge). The key is to understand the primary goal of the fine-tuning process and tailor the approach accordingly.

At this point in the process, we should have a collection of datasets suited for our use case. The next step consists of refining the quality of the samples through rule-based filtering, data duplication, data decontamination, and data quality evaluation.

Rule-based filtering

Rule-based filtering is a systematic approach to data quality control that relies on explicit, predefined rules to evaluate and filter data samples. These rules are typically designed to address common quality issues and can range from simple checks to more complex logical operations. The primary goal of rule-based filtering is to maintain a high standard of data quality by removing samples that do not meet specific criteria.

Length filtering is a straightforward yet effective rule-based filtering technique. This method involves setting thresholds for the acceptable length of responses in the dataset. Extremely short responses often lack sufficient information to be meaningful, while excessively long ones may contain irrelevant or redundant content. It's important to note that the appropriate length thresholds can vary significantly depending on the specific task and domain. For example, a dataset for generating concise

summaries might have a lower maximum threshold compared to one for detailed explanations.

Keyword exclusion is another powerful rule-based filtering technique that focuses on the content of the samples rather than their structure. This method involves creating a list of keywords or phrases associated with low-quality or inappropriate content, and then filtering out any samples that contain these terms. The keyword list can include obvious indicators of low quality, such as profanities or spam-related terms, as well as domain-specific words that might indicate irrelevant or off-topic content. For instance, in a dataset for a professional writing assistant, you might exclude samples containing slang terms or informal expressions that don't align with the intended tone and style.

Format checking is recommended for datasets that include structured data or follow specific formatting requirements. This technique ensures that all samples adhere to the expected format, maintaining consistency and facilitating processing downstream. Format checking can be particularly important for datasets containing code samples, JSON structures, or other formatted text. For example, in a dataset of programming instructions and solutions, you might implement rules to verify that code samples are syntactically correct and follow specified style guidelines.

Rule-based filtering offers significant advantages in preparing instruction datasets. Its speed and efficiency allow for rapid application to large volumes of data, making it highly scalable. The consistency of rule application ensures uniform treatment of data, reducing human error and bias. Furthermore, the explicit definition of filtering criteria provides transparency and interpretability, facilitating easy understanding, auditing, and adjustment. The ability to automate rule-based filtering reduces the need for manual intervention and enables continuous data quality monitoring.

However, rule-based filtering also has limitations that must be considered. Predefined rules may lack the nuance required to capture the full complexity of language and context, potentially leading to the removal of valid but unusual samples. The typically binary nature of rules (pass/fail) may not always align with the nuanced nature of language and instruction quality. Additionally, as data patterns and quality standards evolve, rules need regular review and updates to remain effective. There's also a risk that poorly designed rules could inadvertently introduce or amplify biases in the dataset.

Data deduplication

Dataset diversity is fundamental to training models that can generalize well to new, unseen data. When a dataset contains duplicates or near-duplicates, it can lead to several issues:

- **Overfitting:** Models may memorize specific examples rather than learning general patterns.
- **Biased performance:** Overrepresented data points may skew the model's performance towards certain types of inputs.
- **Inefficient training:** Redundant data can increase training time without providing additional valuable information.

- Inflated evaluation metrics: Duplicate data in test sets may lead to overly optimistic performance estimates.

To deduplicate datasets, we distinguish between exact and fuzzy deduplication. **Exact deduplication** removes identical samples through a straightforward process involving data normalization, hash generation, and duplicate removal. Data normalization standardizes the format of entries, such as converting text to lowercase. Hash generation then creates unique hashes for each entry using algorithms like MD5 or SHA-256. These hashes are compared to find matches, and duplicates are removed, leaving only one instance of each. While effective for identical entries, exact deduplication does not detect near-duplicates or semantically similar content, requiring more advanced techniques for those cases.

The most popular approach to **fuzzy deduplication** is MinHash deduplication. Compared to other fuzzy techniques, it maintains high accuracy while significantly reducing computational complexity. MinHash operates by generating compact representations, or signatures, for each data item. These signatures serve as fingerprints that capture the essence of the data while drastically reducing its dimensionality. In practice, MinHash transforms data items (such as text documents) into sets of shingles, applies multiple hash functions to these sets, and selects the minimum hash values to form signature vectors. These signatures can then be compared using similarity measures like Jaccard similarity to efficiently identify near-duplicates.

In addition to exact and fuzzy deduplication, **semantic similarity** takes a different approach by focusing on the meaning of text for deduplication. This method involves converting words or entire samples into vector representations using various natural language processing techniques. Word embedding models such as Word2Vec, GloVe, and FastText transform individual words into dense vectors, capturing semantic relationships. For more context-aware representations, language models like BERT, sentence transformers, or cross-encoders can generate embeddings for entire sentences or documents. Once these vector representations are obtained, deduplication can be performed by comparing the similarity between vectors. Common similarity measures include cosine similarity or Euclidean distance. Samples with high similarity scores above a predefined threshold can be considered duplicates. For large datasets, clustering techniques may be applied to group similar vectors. Methods like K-means, DBSCAN, or hierarchical clustering can efficiently organize the vector space, allowing for the identification of clusters that represent semantically similar content. Within each cluster, a representative sample can be retained while others are marked as duplicates.

Data decontamination

Data decontamination is the process of ensuring that the training dataset does not contain samples that are identical or highly similar to those in the evaluation or test sets. This step is important for ensuring the quality of the model evaluation and preventing overfitting or memorization of test data.

Data decontamination uses techniques from data deduplication. Exact matching can be used to remove any training samples that are identical to those in the evaluation sets. This can be done using hash functions or

direct string comparisons. Next, we can also use near-duplicate detection methods to identify and remove training samples that are very similar to evaluation samples, even if they are not exactly the same. This often involves techniques like MinHash or computing similarity scores based on n-grams or embeddings.



A simple way to perform data decontamination is to add your evaluation set to the instruction dataset during the data deduplication stage. In this case, we want to ensure that we only remove samples from the instruction dataset, which can be implemented in different ways (only filtering out the first duplicate, recording the indexes of the evaluation samples, etc.). Ideally, you can automatically add your evaluation sets in the data deduplication stage to fully automate this process. This is particularly efficient if you iterate over several versions of custom benchmarks.

Another aspect of data decontamination is filtering out samples that may have been derived from the same source as evaluation data. This can involve checking for overlapping phrases, similar sentence structures, or common metadata. Practitioners may also use provenance tracking (source the data they use) to identify and exclude data from specific sources that are known to be used in evaluation sets.

Data quality evaluation

Data quality evaluation is a critical aspect of machine learning, particularly for LLMs. The process involves assessing various characteristics of datasets, including accuracy, diversity, and complexity. While some aspects like mathematical accuracy can be easily verified using tools such as Python interpreters, evaluating subjective or open-ended content remains challenging.

Traditional methods of data quality assessment include human annotation, which generally provides high accuracy but is resource-intensive. To address scalability issues, machine learning techniques have been developed to automate the evaluation process. These include using LLMs as judges, reward models, and classifiers trained for quality prediction.

The LLM-as-a-judge strategy involves prompting LLMs to evaluate the quality of each sample. This approach has become popular due to its flexibility and ease of use, though it does present some challenges. Different LLMs have different levels of performance across tasks, and their evaluations often align more closely with those of non-experts. With domain-specific datasets, you might want to use domain-specific models instead of better, general-purpose LLMs. Comparative assessment methods (e.g., “Is answer A better than answer B?”) generally outperform absolute scoring approaches (e.g., “Rate answer A between 1 and 4”), though both can be used at scale with sufficient prompt engineering. We recommend iterating through different prompts over a representative subset to manually verify the quality of the responses. *Table 5.2* shows an example of a custom prompt for a judge LLM.

Instruction

You are a data quality evaluator. Your goal is to assess an instruction and its corresponding answer, determining how effectively the answer addresses the given task.

In your evaluation, you will provide feedback detailing the strengths and weaknesses of the answer, followed by a score on a scale of 1 to 4. A score of 1 means that the answer is terrible and irrelevant to the instruction.

A score of 2 means that the answer is not helpful and misses important aspects of the instruction.

A score of 3 means that the answer is helpful but could be improved in terms of relevance, accuracy, and depth.

A score of 4 means that the answer is excellent and fully addresses the task.

Provide your evaluation as follows:

Feedback: (strengths and weaknesses you find relevant)

Score: (number between 1 and 4)

Table 5.2 – Example of LLM-as-a-judge prompt for data quality evaluation

LLM-as-a-judge is known to have several biases. First, it has a position bias in comparative scoring, where the LLM judge favors the first answer. This can be addressed by randomizing the order of answers A and B. In addition, like humans, LLM judges favor long answers. Length normalization techniques can be applied to absolute scoring to mitigate this issue. Finally, LLM judges are known to have intra-model favoritism, meaning that they prefer models from the same family (GPT-4o with GPT-4 and GPT-4o mini, for example). This can be addressed by using several models instead of a single one.

In general, to improve evaluation reliability, strategies such as using multiple LLMs as a jury reduce bias and improve consistency. Leveraging a jury of smaller LLMs can also reduce costs while increasing accuracy and mitigating intra-model favoritism. For specific applications like chatbots, it's advisable to aim for high agreement between LLM judges and human evaluators (around 80%). Simple grading scales (with few-shot prompting) and task-specific benchmarks are also recommended to ensure relevant and interpretable evaluations.

Reward models are another way to re-purpose LLMs for data quality evaluation. The term “reward model” comes from Reinforcement Learning from Human Feedback (RLHF, see *Chapter 6*). They can be broadly defined as models that take an instruction and answer pair and return a score as output. Generally, reward models are created by adding a linear head on top of a decoder-only architecture like Gemma or Llama. They are then trained for this specific purpose, using either reinforcement learning or traditional fine-tuning. *Figure 5.3* shows ArmoRM-Llama3-8B-v0.1's architecture, which adds regression and gating layers on top of a Llama 3 8B model. This model outputs multiple scores to target specific dimensions, such as helpfulness, correctness, coherence, complexity, and verbosity. This allows for a more fine-grained approach to data quality evaluation.

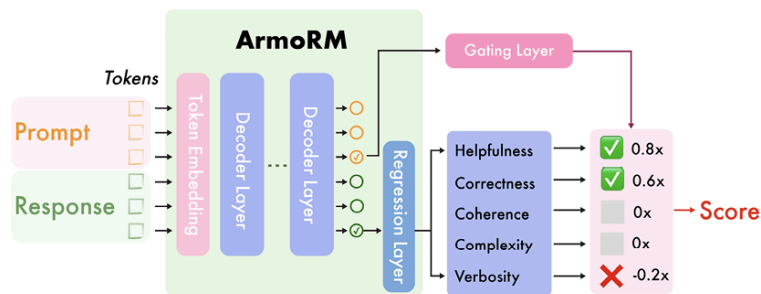


Figure 5.3 – Architecture of RLHFlow/ArmoRM-Llama3-8B-v0.1, based on Llama 3 (Source: <https://doi.org/10.48550/arXiv.2406.12845>)

The Allen Institute for AI's RewardBench leaderboard, hosted on Hugging Face ([allenai/reward-bench](https://huggingface.co/allenai/reward-bench)), is a good resource for comparing different reward models. It combines various types of reward models (generative, classifiers, DPO, etc.) and evaluates them on a curated set of chosen and rejected answers for each instruction. While this task is not directly related to instruction data quality, it is a good resource for finding models capable of differentiating between good and bad answers.

Classifiers or encoder-only models can be trained to perform data quality evaluation. A good example is HuggingFaceFW/fineweb-edu-classifier, a classifier designed to judge the educational value of web pages. This model was designed as a quality filter for pretraining data but a similar approach can be taken to evaluate instruction samples at scale. In practice, fineweb-edu-classifier adds a classification head to an embedding model (Snowflake/snowflake-arctic-embed-m) and trains it for 20 epochs on 450,000 samples that are annotated by Llama 3 70B Instruct.

This approach relies on encoder-only models, which are both smaller and better suited to classification tasks. Thanks to their low number of parameters, these models are faster to run and can scale to millions of samples. However, they are not as accurate as bigger models, particularly for complex reasoning tasks where they lack the ability to capture nuances. At smaller scale, encoder-only models are still valuable to filter out outliers or as part of an automated data pipeline, which requires faster processing.

Data exploration

Data exploration is a continuous process that requires practitioners to become familiar with the training data. It involves both manual inspection and automated analysis, each playing a crucial role in understanding the dataset's characteristics, strengths, and potential shortcomings.

Manual dataset exploration, though time-consuming, is an important step. It reveals errors and inconsistencies that automated processes might miss, including formatting issues, data entry mistakes, incoherent reasoning, and factual inaccuracies. This process provides qualitative insights into the dataset's content and style. To enhance efficiency, researchers can employ techniques like stratified sampling (selecting diverse samples), systematic review (using a criteria checklist), and collaborative review (involving multiple reviewers).

Figure 5.4 shows an example with Argilla, a collaborative platform for manual data quality evaluation and exploration.

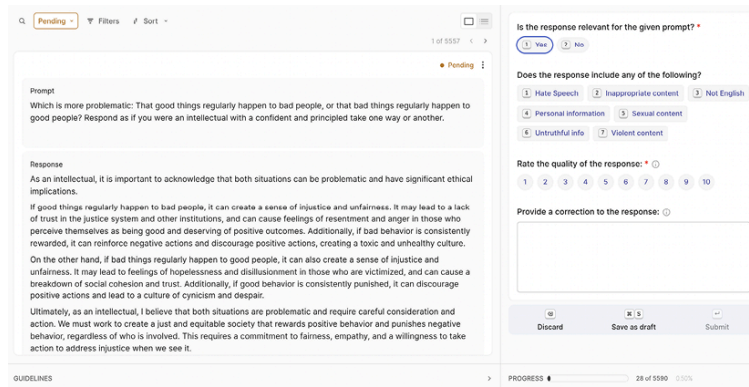


Figure 5.4 – Argilla’s interface for collaborative data quality evaluation and exploration

Statistical analysis is a complementary technique that reveals vocabulary diversity, potential biases, and concept representation. This process utilizes natural language processing libraries like NLTK or spaCy for tokenization and analysis of large text volumes. Visualization tools such as Matplotlib or Seaborn create histograms and word clouds, enabling intuitive pattern recognition. These techniques provide insights into dataset composition, language breadth, and possible cultural or contextual preferences, which can influence model outputs.

Topic clustering automatically groups similar documents or pieces of text together, revealing underlying themes and patterns within the data. This process is especially important for understanding the content of large text corpora, identifying trends, and organizing information in a meaningful way. It is often associated with data visualization, with figures that show clusters of similar samples.

Let’s consider the task of building an instruction dataset about various programming languages. You have collected a vast corpus of programming-related text from online forums, documentation, and tutorials. First, topic clustering can help identify the distinct programming languages present in the dataset (Python, JavaScript, etc.). Second, within each language cluster, you can further identify sub-topics like `error handling`, `data structures`, and `web frameworks`. This allows a balanced representation of each language and sub-topic in the corpus.

This makes sure that each topic is correctly covered for each programming language.

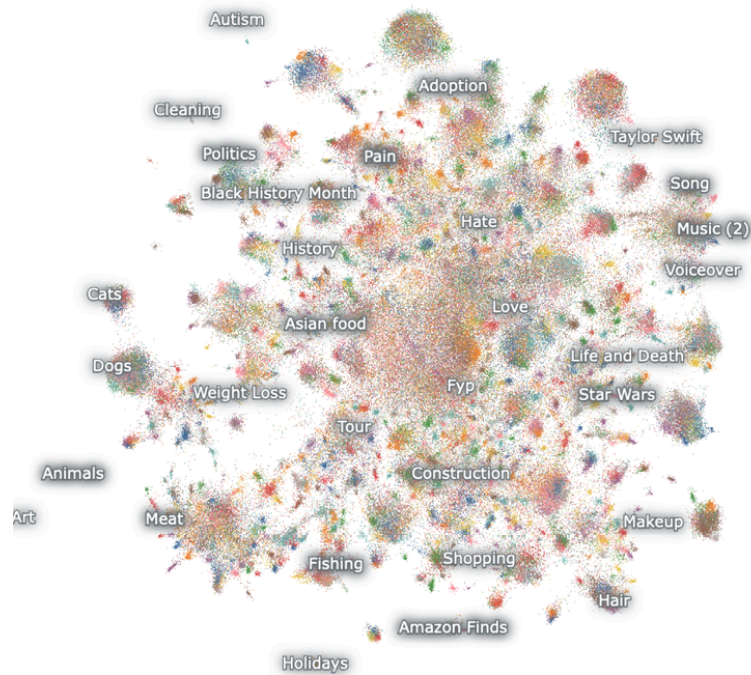


Figure 5.5 – Representation of the historical TikTok dataset made with Nomic Atlas

Several tools are available for performing topic clustering, each with its own strengths and approaches. For example, Hugging Face's text-clustering provides a simple pipeline with sentence transformers for embedding text into vector space, UMAP for dimensionality reduction, and DBSCAN for clustering. It also automatically labels clusters using an LLM and can output visualizations. Nomic Atlas (see Figure 5.5), BunkaTopics, and Lilac are alternatives proposing similar approaches with additional features.

Data generation

When the available instruction datasets are not sufficient, creating custom data becomes necessary. This is particularly relevant for specialized applications where publicly available data is scarce.

Additionally, it serves as a method to augment underrepresented areas in a dataset, like insufficient examples of JavaScript error-handling techniques in our previous example. While data can be generated manually by individuals or through crowdsourcing, these approaches often incur significant costs and time investments. Synthetic data generation using LLMs offers a more efficient and scalable alternative. This method, when combined with well-designed prompt engineering, can produce high-quality data at a much larger scale, effectively addressing the limitations of manual data creation processes.

The process of synthetic data generation typically begins with the preparation of a set of carefully designed prompts (sometimes called taxonomy). These serve as the foundation for generating new, diverse examples. Five seed prompts used in the original Alpaca dataset can be seen in Table 5.3. The quality of synthetically generated data largely depends on the prompts and techniques used in the generation process. Well-crafted prompts can guide the language model to produce diverse, relevant, and high-quality instruction-response pairs. These prompts often include

specific instructions, examples, and constraints to ensure the generated data aligns with the desired format and content.

Seed instructions

- Is there anything I can eat for breakfast that doesn't include eggs, yet includes protein, and has roughly 700-1000 calories?
- What is the relation between the given pairs? Input: Night : Day :: Right : Left
- Generate a one-sentence description for each of the following people. Input: -Barack Obama\n- Elon Musk\n- Taylor Swift
- Describe a situation in which the given stereotype can harm you. Input: All Asians are smart!
- Generate an appropriate subjective title for the following email: Input: "Hi [person name],\n\nI'm writing to ask you if you are happy to be a panelist in our workshop on multimodality at CVPR. The workshop will be held on June 20, 2023. \n\nBest,\n[my name]"

Table 5.3 – Examples of seed prompts used in the original Alpaca dataset

Many synthetic data generation pipelines incorporate multiple steps to ensure data quality. This may include generating an initial set of questions or instructions, followed by generating corresponding answers or responses. Some systems also implement validation steps, where another model or set of rules checks the generated pairs for accuracy, relevance, and adherence to specified criteria.

An important aspect of synthetic data generation is the ability to control various attributes of the generated data. This includes factors such as the complexity of the instructions, the length of the responses, the tone or style of the language used, and the specific topics or domains covered. By fine-tuning these parameters, it's possible to create datasets that are tailored to specific training objectives or that complement existing datasets in targeted ways. Structured generation using libraries like Outlines can also be beneficial to adhere to specific formats.

Furthermore, synthetic data generation can be particularly useful for addressing biases and gaps in existing datasets. By carefully designing the generation process, it's possible to create more balanced and inclusive datasets that represent a wider range of perspectives, topics, and language styles. This can help in training LLMs that are more equitable and capable of serving diverse user bases.

However, synthetic data generation also comes with challenges. One primary concern is the potential for the generated data to inherit biases or errors from the underlying language model used for generation. To mitigate this, many approaches incorporate human oversight, diverse prompts, and additional filtering mechanisms to ensure the quality and appropriateness of the generated data.

Another consideration is the need for the generated data to be sufficiently diverse and challenging. If the synthetic data is too simplistic or repetitive, it may not provide the level of complexity required to train a robust LLM. Advanced techniques in synthetic data generation often focus on

creating varied and nuanced instruction-response pairs that can push the boundaries of what the model can learn.

Data augmentation

In this context, data augmentation refers to the process of increasing both the quantity and the quality of data samples. Unlike data generation, we use pre-existing instruction samples as inputs in this stage. While it is possible to upsample pairs of instructions and answers, data augmentation is mostly used to increase the quality of existing samples. In particular, it focuses on two aspects: diversity and complexity.

A pioneering approach in this field is the Evol-Instruct method, which uses LLMs to evolve simple instructions into more qualitative ones. The evolved instructions can then be used to generate answers using powerful LLMs. This method employs two main strategies: in-depth and in-breadth evolving.

In-depth evolving focuses on enhancing the complexity of existing instructions. It includes several techniques:

- **Constraints:** It involves introducing additional requirements or limitations to the original instruction, making it more challenging to fulfill.
- **Deepening:** Instead of shallow questions, it tries to find more deep questions, requiring more comprehensive responses.
- **Concretizing:** It replaces general concepts with more specific ones, adding detail and precision to the instruction.
- **Increasing reasoning steps:** It modifies instructions to explicitly request multiple-step reasoning, promoting more complex problem-solving.
- **Complicating input:** This involves adding more complex data formats or structures to the instruction, such as XML, JSON, or code snippets.

In-breadth evolving, on the other hand, aims to expand the diversity of the instruction dataset. It generates entirely new instructions inspired by existing ones, focusing on creating more rare or long-tailed examples within the same domain.

As an example of concrete implementation, in-depth evolving can be automated with the following prompt, from the AutoEvol paper. You simply need to provide the instruction you want to evolve as input, and a powerful model like GPT-4o will return a more complex version of the original instruction.

You are an Instruction Rewriter that rewrites the given #Instruction# into a more complex version. Please follow the steps below to rewrite the given “#Instruction#” into a more complex version.

- Step 1: Please read the “#Instruction#” carefully and list all the possible methods to make this instruction more complex (to make it a bit harder for well-known AI assistants such as ChatGPT and GPT4 to handle). Please do not provide methods to
- change the language of the instruction!
- Step 2: Please create a comprehensive plan based on the #Methods List# generated in Step 1 to make the #Instruction# more complex. The plan should include several methods from the #Methods List#.

- Step 3: Please execute the plan step by step and provide the #Rewritten Instruction#. #Rewritten Instruction# can only add 10 to 20 words into the “#Instruction#”.
- Step 4: Please carefully review the #Rewritten Instruction# and identify any unreasonable parts. Ensure that the #Rewritten Instruction# is only a more complex version of the #Instruction#. Just provide the #Finally Rewritten Instruction# without any explanation.

Please reply strictly in the following format:

Step 1 #Methods List#:

Step 2 #Plan#:

Step 3 #Rewritten Instruction#:

Step 4 #Finally Rewritten Instruction#:

#Instruction#:

{Instruction}

Table 5.4 – Evol LLM prompt from the “Automatic Instruction Evolving for Large Language Models” paper by Zeng et al. (2024)

The UltraFeedback method is another innovative approach, focused on answer quality instead of instruction quality. It employs AI feedback to enhance the quality and diversity of model responses. Unlike Evol-Instruct, which evolves instructions, UltraFeedback uses a large pool of diverse instructions and models to generate a wide range of responses.

It then leverages advanced language models like GPT-4 to provide detailed critiques and numerical scores for these responses across multiple dimensions such as instruction-following, truthfulness, honesty, and helpfulness.

Based on these ideas, you can create your own augmentation techniques to create a more challenging and diverse instruction dataset. By refining and evolving existing instructions and answers, the resulting dataset can better train models to handle complex, multi-step tasks, and improve their performance across a wider range of applications.

Creating our own instruction dataset

In this section, we will create our own instruction dataset based on the crawled data from *Chapter 3*. To create a high-quality instruction dataset, we need to address two main issues: the unstructured nature of our data and the limited number of articles we can crawl.

This unstructured nature comes from the fact that we are dealing with raw text (articles), instead of pairs of instructions and answers. To address this issue, we will use an LLM to perform this transformation. Specifically, we will employ a combination of backtranslation and rephrasing. Backtranslation refers to the process of providing the expected answer as output and generating its corresponding instruction. However, using a chunk of text like a paragraph as an answer might not always be appropriate. This is why we want to rephrase the raw text to ensure we're outputting properly formatted, high-quality answers. Additionally, we can ask the model to follow the author's writing style to stay close to the original paragraph. While this process involves extensive prompt engineering, it can be automated and used at scale, as we will see in the following implementation.

Our second issue regarding the limited number of samples is quite common in real-world use cases. The number of articles we can retrieve is limited, which constrains the size of the instruction dataset we are able to create. In this example, the more samples we have, the better the model becomes at imitating the original authors. To address this problem, we will divide our articles into chunks and generate three instruction-answer pairs for each chunk. This will multiply the number of samples we create while maintaining diversity in the final dataset. For simplicity, we will do it using OpenAI's GPT-4o-mini model, but you can also use open-source models.

However, LLMs are not reliable when it comes to producing structured output. Even when given specific templates or instructions, there's no guarantee that the model will consistently adhere to them. This inconsistency often necessitates additional string parsing to ensure the output meets the desired format.

To simplify this process and ensure properly structured results, we can employ structured generation techniques. Structured generation is an effective method to force an LLM to follow a predefined template, such as JSON, pydantic classes, or regular expressions. In the following, we will use OpenAI's JSON mode feature, which provides a more robust way to return valid JSON objects and reduce the need for extensive post-processing.

Based on this description, the following figure summarizes every step of the synthetic data pipeline we want to build.

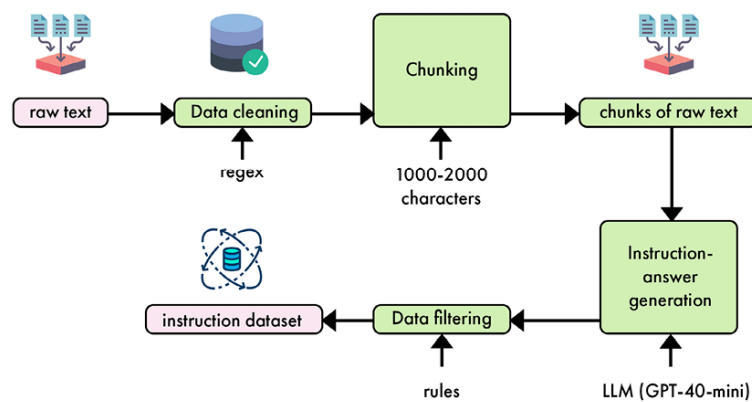


Figure 5.6 – Synthetic data generation pipeline from raw text to instruction dataset

Let's now implement it in Python. You can implement it as part of the LLMops pipeline, or as a standalone script:

1. We want to make sure that the following libraries are installed. The OpenAI library will allow us to interact with a model to generate the instruction data, and datasets will format it into a Hugging Face-compatible format. The tqdm library is installed to visualize the progress during the data generation process.

```

openai==1.37.1
datasets==2.20.0
tqdm==4.66.4

```

2. We import all the required libraries as follows.

```
import concurrent.futures
import json
import random
import re
from concurrent.futures import ThreadPoolExecutor
from typing import List, Tuple
from datasets import Dataset
from openai import OpenAI
from pydantic import BaseModel, Field
from tqdm.auto import tqdm
```



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (1) to quickly copy code into your coding environment, or click the **Explain** button (2) to get the AI assistant to explain a block of code to you.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting <https://www.packtpub.com/unlock/9781836200079>.

3. The raw data we have is a JSON file. We create a Hugging Face dataset from this JSON file by extracting specific fields from each article: `id`, `content`, `platform`, `author_id`, `author name`, and `link`.

```
def load_articles_from_json(file_path: str) -> Dataset:
    with open(file_path, "r") as file:
        data = json.load(file)
    return Dataset.from_dict(
        {
            "id": [item["id"] for item in data["artifact_data"]],
            "content": [item["content"] for item in data["artifact_data"]],
            "platform": [item["platform"] for item in data["artifact_data"]],
            "author_id": [item["author_id"] for item in data["artifact_data"]],
            "author_full_name": [item["author_full_name"] for item in data["artifact_data"]],
            "link": [item["link"] for item in data["artifact_data"]],
        }
    )
```

If we simply load our dataset as a pandas dataframe, it returns the following table.

id	content	platform	author_id	author_full_name	link
----	---------	----------	-----------	------------------	------

0	ab2f9e2e-5459-4dd6-97d6-c291de4a7093	The Importance of Data Pipelines in the Era of...	medium	e6b945ba-6a9a-4cde-b2bf-0890af79732b	Alex Vesa	http://medium.com/decode-in-gml/these-important-...
1	ccfe70f3-d324-40b6-ba38-86e72786dcf4	Change Data Capture: Enabling Event-Driven Arc...	medium	e6b945ba-6a9a-4cde-b2bf-0890af79732b	Alex Vesa	http://medium.com/decode-in-gml/these-3rd-out-of-1...
2	4c9f68ae-ec8b-4534-8ad5-92372bf8bb37	The Role of Feature Stores in Fine-Tuning LLMs...	medium	e6b945ba-6a9a-4cde-b2bf-0890af79732b	Alex Vesa	http://medium.com/decode-in-gml/these-role-of-fea

...	...	...	...	...	...	...
73	68795a4d-26c2-43b7-9900-739a80b9b7dc	DML: 4 key ideas you must know to train an LLM...	de-cod-ingm-l.sub-stack.-com	1519b1d1-1a5d-444c-a880-926c9e-b6539e	Paul Iusztin	http://dml-4-key-id.
74	d91b17c0-05d8-4838-bf61-e2abc1573622	DML: How to add real-time monitoring & metrics...	de-cod-ingm-l.sub-stack.-com	1519b1d1-1a5d-444c-a880-926c9e-b6539e	Paul Iusztin	http://dml-how-to-a..
75	dcf55b28-2814-4480-a18b-a77d01d44f5f	DML: Top 6 ML Platform Features You Must Know ...	de-cod-ingm-l.sub-stack.-com	1519b1d1-1a5d-444c-a880-926c9e-b6539e	Paul Iusztin	http://dml-stack.com/p/

[dml](#)
[=](#)
[top](#)
[-6-](#)
[ml](#)
[..](#)

1. If we inspect the content of some articles a little further, we realize that some of them have special characters and redundant whitespaces. We can clean this with a simple regex.

First, we use `[\w\s.,!?' ]` to remove non-alphanumeric characters except for apostrophes, periods, commas, exclamation marks, and question marks. Then, we use `\s+` to replace multiple consecutive whitespace characters with a single space. Finally, we implement `strip()` to remove any leading or trailing whitespace.

```
def clean_text(text):
    text = re.sub(r"[\w\s.,!?' ]", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

5. Now that we can load our articles, we need to chunk them before turning them into pairs of instructions and answers. Ideally, you would want to use headlines or paragraphs to produce semantically meaningful chunking.

However, in our example, like in the real world, raw data tends to be messy. Due to improper formatting, we cannot extract paragraphs or headlines for every article in our raw dataset. Instead, we will extract sentences using a regex to get chunks between 1,000 and 2,000 characters. This number can be optimized depending on the density of the information contained in the text.

The `extract_substrings` function processes each article in the dataset by first cleaning the text and then using a regex to split it into sentences. It then builds chunks of text by concatenating these sentences until each chunk is between 1,000 and 2,000 characters long.

```
def extract_substrings(dataset: Dataset, min_length: int = 1000, max_length: int = 2000):
    extracts = []
    sentence_pattern = r"(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|?|!)\s"
    for article in dataset["content"]:
        cleaned_article = clean_text(article)
        sentences = re.split(sentence_pattern, cleaned_article)
        current_chunk = ""
        for sentence in sentences:
            sentence = sentence.strip()
            if not sentence:
                continue
            if len(current_chunk) + len(sentence) <= max_length:
                current_chunk += sentence + " "
            else:
                if len(current_chunk) >= min_length:
                    extracts.append(current_chunk.strip())
                    current_chunk = sentence + " "
                if len(current_chunk) >= min_length:
```

```
extracts.append(current_chunk.strip())
return extracts
```

6. Next, we want to create instruction-answer pairs from the extracted chunks of text. To manage these pairs effectively, we introduce the `InstructionAnswerSet` class. This class allows us to create instances directly from JSON strings, which is useful when parsing the output from the OpenAI API.

```
class InstructionAnswerSet:
    def __init__(self, pairs: List[Tuple[str, str]]):
        self.pairs = pairs
    @classmethod
    def from_json(cls, json_str: str) -> 'InstructionAnswerSet':
        data = json.loads(json_str)
        pairs = [(pair['instruction'], pair['answer'])
                  for pair in data['instruction_answer_pairs']]
        return cls(pairs)
    def __iter__(self):
        return iter(self.pairs)
```

7. Now that we have a set of extracts from the articles with a reasonable length, we can use an LLM to transform them into pairs of instructions and answers. Note that this step is model-agnostic and can be implemented with any open-source or closed-source model. Because this output is grounded in the context we provide, it doesn't require complex reasoning or high-performing models.

For convenience, we will use GPT-4o mini in this example. This choice is motivated by the low cost and good performance of this model. Prompt engineering is the most important aspect of this data transformation stage and requires several iterations to produce the expected outputs. We recommend starting with simple prompts and adding complexity when required to be more accurate, modify the style, or output multiple responses.

In our example, we want to create instructions like “Write a paragraph about X topic” and corresponding answers that are factual and imitate the writer's style. To implement this, we need to provide an extract that will ground the model's responses. For efficiency, we also choose to generate five instruction-answer pairs for each extract. Here's the beginning of our function for instruction generation, including our prompt.

```
def generate_instruction_answer_pairs(
    extract: str, client: OpenAI
) -> List[Tuple[str, str]]:
    prompt = f"""Based on the following extract, generate five instruction-answer pairs. Each instruction must ask to write about a specific topic contained in the context. Each answer must provide a relevant paragraph based on the information found in the context. Only use concepts from the context to generate the instructions. Instructions must never explicitly mention a context, a system, a course, or an external source. Instructions must be self-contained and general. Answers must imitate the writing style of the context. Example instruction: Explain the concept of an LLM Twin. Example answer: An LLM Twin is essentially an AI character that mimics your writing style. It's designed to write just like you by incorporating these elements into a language model. The idea is to create a digital replica of your writing habits using advanced AI technology. Provide your response in JSON format with the following structure:
```

```

{{
  "instruction_answer_pairs": [
    {"instruction": "...", "answer": "..."},
    ...
  ]
}}
Extract:
{extract}
"""

```

8. In addition to the user prompt, we can also specify a system prompt to guide the model into generating the expected instructions. Here, we repeat our high-level task in the system prompt.

The concatenation of the system and user prompts is fed to the OpenAI API, using the GPT-4o mini model in JSON mode and a maximum of 1,200 tokens in the answer. We also use a standard temperature of 0.7 to encourage diverse responses. The generated text is directly parsed using the `InstructionAnswerSet` class to return pairs of instructions and answers.

```

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "system", "content": "You are a helpful assistant who \
generates instruction-answer pairs based on the given context. \
Provide your response in JSON format.",
        },
        {"role": "user", "content": prompt},
    ],
    response_format={"type": "json_object"},
    max_tokens=1200,
    temperature=0.7,
)
# Parse the structured output
result = InstructionAnswerSet.from_json(completion.choices[0].message.content)
# Convert to list of tuples
return result.pairs

```

9. Let's create a main function to automate the process. It extracts substrings from the input dataset, then uses concurrent processing via Python's `ThreadPoolExecutor` to efficiently generate instruction-answer pairs for each extract. We use a default `max_workers` value of 4 because higher values tend to exceed OpenAI's rate limits, potentially causing API request failures or throttling.

```

def create_instruction_dataset(
    dataset: Dataset, client: OpenAI, num_workers: int = 4
) -> Dataset:
    extracts = extract_substrings(dataset)
    instruction_answer_pairs = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=num_workers) as executor:
        futures = [executor.submit(generate_instruction_answer_pairs, extract, c
                                for extract in extracts
        ]
        for future in tqdm(concurrent.futures.as_completed(futures), total=len(f
        ):
            instruction_answer_pairs.extend(future.result())
    instructions, answers = zip(*instruction_answer_pairs)
    return Dataset.from_dict(

```



```

    {"instruction": list(instructions), "output": list(answers)}
)

```

10. We can create our instruction dataset by calling this function. Running it over the raw data with GPT-4o mini costs less than 0.5\$.
11. We can now create a main function to orchestrate the entire pipeline. It loads the raw data, creates the instruction dataset, splits it into training and testing sets, and pushes the result to the Hugging Face Hub.

```

def main(dataset_id: str) -> Dataset:
    client = OpenAI()
    # 1. Load the raw data
    raw_dataset = load_articles_from_json("cleaned_documents.json")
    print("Raw dataset:")
    print(raw_dataset.to_pandas())
    # 2. Create instructiondataset
    instruction_dataset = create_instruction_dataset(raw_dataset, client)
    print("Instruction dataset:")
    print(instruction_dataset.to_pandas())
    # 3. Train/test split and export
    filtered_dataset = instruction_dataset.train_test_split(test_size=0.1)
    filtered_dataset.push_to_hub("mlabonne/llmtwin")
    return filtered_dataset

Dataset({
  features: ['instruction', 'output'],
  num_rows: 3335
})

```

We obtained 3,335 pairs with this process. You can find our version of the dataset at <https://huggingface.co/datasets/mlabonne/llmtwin>. The Hugging Face Hub provides a convenient dataset viewer (see Figure 5.7) to explore instructions and answers and make sure that there are no obvious mistakes in these samples.

Due to the small size of the dataset, there is no need for comprehensive exploration and topic clustering.

instruction string · lengths	output string · lengths
57-67 26.2%	561-628 21.7%
Explain how list comprehensions improve iteration speed in Python.	List comprehensions serve as a highly efficient mechanism for iterating over lists in Python, enabling operations to be expressed in a single line. This method eliminates the need to explicitly call the .append method during iteration, which contributes to its speed. Python processes list comprehensions similarly to traditional for loops at the bytecode level, yet the overhead of function calls is reduced, resulting in a performance boost. As evidenced by benchmarks, list comprehensions can achieve speeds that are 200 times faster than conventional looping methods.
Define what an LLM Twin is and its purpose.	An LLM Twin is an AI-driven character designed to replicate your own writing style, personality, and voice. By leveraging large language...
Describe how audio narrations can enhance the reading experience.	Audio narrations can significantly enhance the reading experience by adding an auditory dimension to the text. This feature allows readers to...
Describe the role of a vector database in embedding storage.	A vector database serves as a crucial repository for storing embeddings along with their associated metadata, such as the embedded text, source...
Explain the concept of quantization in the context of LLMs.	Quantization in the context of LLMs refers to the process of reducing the precision of the numbers used to represent the model's weights. Thi...

Figure 5.7 – The mlabonne/llmtwin instruction dataset on the Hugging Face Hub

As seen in the previous section, we could refine this instruction dataset by increasing the diversity and complexity of our samples. More advanced prompt engineering could also increase the quality of the generated data by providing examples of the expected results, for instance. Finally, quality evaluation could help filter out low-quality samples by reviewing them individually. For conciseness and simplicity, we will keep a straightforward approach for this instruction dataset and explore more advanced methods in Chapter 6 when we create a preference dataset.

In the next section, we will introduce SFT techniques, as well as related concepts.

Exploring SFT and its techniques

SFT consists of re-training pre-trained models on a smaller dataset composed of pairs of instructions and answers. The goal of SFT is to turn a base model, which can only perform next-token prediction, into a useful assistant, capable of answering questions and following instructions. SFT can also be used to improve the general performance of the base model (general-purpose SFT), instill new knowledge (e.g., new languages, domains, etc.), focus on specific tasks, adopt a particular voice, and so on.

In this section, we will discuss when to use fine-tuning and explore related concepts with storage formats and chat templates. Finally, we will introduce three popular ways of implementing SFT: full-finetuning, **Low-Rank Adaptation (LoRA)** and **Quantization-aware Low-Rank Adaptation (QLoRA)**.

When to fine-tune

In most scenarios, it is recommended to start with prompt engineering instead of directly fine-tuning models. Prompt engineering can be used with either open-weight or closed-source models. By using techniques like few-shot prompting or **retrieval augmented generation (RAG)**, numerous problems can efficiently be tackled without SFT. Prompt engineering also allows us to build a robust evaluation pipeline, which measures metrics like accuracy, but also cost and latency. If these results do not match the requirements, we can explore the possibility of creating an instruction dataset, as illustrated in the previous section. If enough data is available, fine-tuning becomes an option.

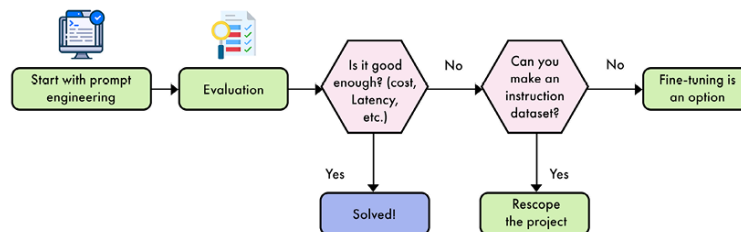


Figure 5.8 – Basic flowchart to determine when fine-tuning is an option on a technical level

Beyond these technical considerations, SFT answers common needs in terms of control (“know your data”) and customizability (the fine-tuned model is unique). Instead of building applications around a chatbot, fine-tuning allows developers to create more diverse interactions with LLMs, like tool analytics, moderation, and additional context. Note that if we focus on open-weight models in this book, several LLM providers offer automated fine-tuning services. While they don’t offer the same level of control and customizability as managing your own fine-tuning pipeline, it can be an interesting trade-off in specific scenarios (e.g., limited resources in terms of machine learning engineering).

Despite these advantages, fine-tuning also has limitations. It is generally understood that SFT leverages pre-existing knowledge in the base model’s weights and refocuses the parameters for a specific purpose. This has sev-

eral implications. First of all, knowledge that is too distant from what has been learned in the pre-training set (such as an unknown or rare language) can be difficult to learn effectively.

Even worse, a study showed that fine-tuning a model on new knowledge could result in more frequent hallucinations. Depending on the SFT technique that is used, we’re also at risk of erasing knowledge that was present in the base model (a common issue referred to as “catastrophic forgetting”).

Instruction dataset formats

Instruction datasets are stored in a particular format to organize instructions and answers. Typically, each sample in the dataset can be represented as a Python dictionary, where keys are prompt types like `system`, `instruction`, `output`, and values corresponding to the actual text. The three most standard formats are Alpaca, ShareGPT, and OpenAI. The following table shows how these data formats are generally organized.

Name	JSONL format
Alpaca	<pre>{“instruction”: “...”, “input”: “...”, “output”: “...”} {“instruction”: “...”, “output”: “...”}</pre>
ShareGPT	<pre>{“conversations”: [{“from”: “...”, “value”: “...”, ...}]}</pre>
OpenAI	<pre>{“conversations”: [{“role”: “...”, “content”: “...”, ...}]}</pre>
OASST	<pre>{“INSTRUCTION”: “...”, “RESPONSE”: “...”}</pre>
Raw text	<pre>{“text”: “...”}</pre>

Table 5.5 – Examples of instruction data storage format

Note that for Alpaca, the “`input`” key is optional. The content of the “`input`” key is only appended to the content of the “`instruction`” key when it exists. We also added the “`raw text`” data format to show that SFT is not inherently different from pre-training. If you choose to re-train a model on raw text, this is a type of fine-tuning generally called “continual pre-training.”

The dataset we created in the previous section has two columns (“`instruction`” and “`output`”) and corresponds to the Alpaca format. Alpaca is sufficient for single-turn instructions and answers, which means it is limited to one instruction and one answer. When you want to process conversations (multiple instructions and answers), formats like ShareGPT or OpenAI are a better fit. By storing each message as a dictionary in a list, they can represent an arbitrarily long conversation in each sample.

The choice of single-turn and multi-turn conversations directly impacts the storage type and depends on the end use case.

Chat templates

Once the instruction-answer pairs are parsed from the dataset format, we want to structure them in a chat template. Chat templates offer a unified way to present the instructions and answers to the model.

In general, they also include special tokens to identify the beginning and the end of a message, or who is the author of the message. Since base models are not designed to follow instructions, they don't have a chat template. This means that you can choose any template when you fine-tune a based model. If you want to fine-tune an instruct model (not recommended), you need to use the same template or it might degrade your performance.

Like instruction dataset formats, there are different chat templates: ChatML, Llama 3, Mistral, and many others. In the open-source community, the ChatML template (originally from OpenAI) is a popular option. It simply adds two special tokens (`<|im_start|>` and `<|im_end|>`) to indicate who is speaking. To give you an example, here is what we obtain when we apply the ChatML template to the instruction-answer pair shown in *Table 5.1*:

```
<|im_start|>system
You are a helpful assistant, who always provide explanation. Think like you are an
<|im_start|>user
Concepts: building, shop, town
Write a sentence that includes all these words.<|im_end|>
<|im_start|>assistant
In our little town, there is a shop inside a big building where people go to buy t
```

Table 5.6 – Sample from Table 5.1 with the ChatML chat template

As you can see, we still have three distinct parts: system, user, and assistant. Each part starts with the `<|im_start|>` token and ends with `<|im_end|>`. The current speaker is identified by a string (like “`system`”) instead of a special token. This is the exact string that is tokenized and used as input by the model during fine-tuning.

However, during inference, we can't provide the expected answer. In this case, we provide the system and user part as shown in *Figure 5.6*, and prompt the model to answer by adding `<|im_start|>assistant\n`.

Because the model has been fine-tuned with this template, it understands that the next tokens should be an answer relevant to the user instruction and guided by the system prompt. This is how fine-tuned models acquire instruction-following capabilities.

A common issue with chat templates is that every single whitespace and line break is extremely important. Adding or removing any character would result in a wrong tokenization, which negatively impacts the performance of the model. For this reason, it is recommended to use reliable templates like Jinja, as implemented in the Transformers library. *Table 5.7* shows a few examples of such templates, including Alpaca, which is both the name of an instruction dataset format and a chat template.

Name **Jinja template****Alpaca**

```
### Instruction: What is the capital of France?
### Response: The capital of France is Paris.<EOS>
```

ChatML

```
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
The capital of France is Paris.<|im_end|>
```

Llama

3

```
<|begin_of_text|><|start_header_id|>user<|end_header_id|>
What is the capital of France?<|eot_id|><|start_header_id|>assistant<|enc
The capital of France is Paris.<|eot_id|>
```

Phi-3

```
<|user|>
What is the capital of France?<|end|>
<|assistant|>
The capital of France is Paris.<|end|>
```

Gemma

```
<bos><start_of_turn>user
What is the capital of France?<end_of_turn>
<start_of_turn>model
The capital of France is Paris.<end_of_turn>
```

Table 5.7 – Example of common chat templates

Jinja implements loops and conditions, which allow the same template to be used for training and inference (`add_generation_prompt`).

Parameter-efficient fine-tuning techniques

While many techniques exist in the literature, SFT has converged on three main techniques: full fine-tuning, LoRA, and QLoRA. We will introduce each technique individually, and weigh their pros and cons depending on your use cases.

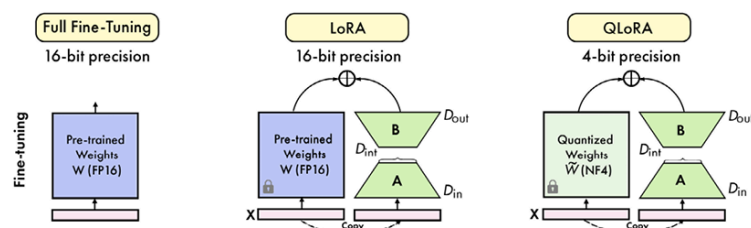


Figure 5.9 – Architectural differences of the three main SFT techniques at the module level

Full fine-tuning

Full fine-tuning refers to the most straightforward SFT technique, consisting of re-training every parameter in the base model. Like pre-training, SFT uses next-token prediction as its training objective. This means that the previously discussed structure of the dataset can be seen as the main difference between continual pre-training and full fine-tuning.

This method often provides the best results but requires significant computational resources. Memory usage depends on several factors, including model size, training techniques, and optimization methods. At its simplest, using a single-GPU setting, the memory required can be estimated using the following formula:

$$\text{Memory} = \text{Parameters} + \text{Gradients} + \text{Optimizer States} + \text{Activations}$$

For a basic setup using **32-bit floating point (fp32)** precision, we can estimate:

- **Parameters:** Learnable weights and biases within a neural network. In a large language model, these are typically the weights in the attention mechanisms, feed-forward layers, and embedding layers. Cost: 4 bytes/parameter (FP32) or 2 bytes/parameter (FP16/BF16).
- **Gradients:** Gradients are the partial derivatives of the loss function with respect to each model parameter. They indicate how much each parameter should be adjusted to minimize the loss. During training, gradients are computed for each parameter through backpropagation and are used to update the model parameters. Cost: 4 bytes/parameter.
- **Optimizer states:** Optimizer states are additional values maintained by optimization algorithms like Adam or AdamW. These typically include running averages of past gradients and past squared gradients for each parameter. They help in adapting the learning rate for each parameter and navigating the loss landscape more effectively. For instance, Adam maintains two additional values (momentum and variance) per parameter. Cost: 8 bytes/parameter (for Adam optimizer).
- **Activations:** Activations are the intermediate outputs of each layer in the neural network during the forward pass. For transformer-based models, this includes the outputs of attention mechanisms, feed-forward layers, and normalization layers. Activations need to be kept in memory during the forward pass to compute gradients in the backward pass, unless techniques like activation checkpointing are used. Cost: variable, but often negligible for small batch sizes.

This gives us a baseline of 16 bytes per parameter. This translates into 112 GB of VRAM for a 7 B model and 1,120 GB for a 70 B model. However, this is often an underestimate, as it doesn't account for additional memory needed for activations, temporary buffers, and overhead from various training techniques.

Several techniques can be employed to reduce memory usage during LLM fine-tuning. Model parallelism spreads the workload across multiple GPUs, though it adds some overhead. Gradient accumulation enables larger effective batch sizes without proportional memory increase. Memory-efficient optimizers like 8-bit Adam can reduce the footprint of optimizer

states. Activation checkpointing trades computation for memory by recalculating certain activations. When combined, these techniques can significantly lower memory usage. For instance, using mixed precision with model parallelism might reduce costs to around 14-15 bytes per parameter, compared to the 16-byte baseline. However, memory requirements remain substantial for large models even with these optimizations.

In addition, full fine-tuning directly modifies the pre-training weights, which makes it destructive by nature. If training doesn't behave as expected, it might erase previous knowledge and skills – a phenomenon referred to as “catastrophic forgetting.” The same phenomenon can happen with continual pre-training, which generally makes these techniques more difficult to use. Due to this additional complexity and its high computational requirements, parameter-efficient techniques are often preferred to full fine-tuning to create task and domain-specific models.

LoRA

LoRA is a parameter-efficient technique for fine-tuning LLMs. Developed to address the computational challenges associated with adapting massive neural networks, LoRA has quickly become a cornerstone technique in LLM fine-tuning.

The primary purpose of LoRA is to enable the fine-tuning of LLMs with significantly reduced computational resources. This is achieved by introducing trainable low-rank matrices that modify the behavior of the model without changing its original parameters. The key advantages of LoRA include:

- Dramatically reduced memory usage during training
- Faster fine-tuning process
- Preservation of pre-trained model weights (non-destructive)
- Ability to switch between tasks efficiently by swapping LoRA weights

These benefits have made LoRA particularly attractive for researchers and developers working with limited computational resources, effectively democratizing the process of LLM fine-tuning.

At its core, LoRA employs a low-rank decomposition technique to update model weights efficiently. Instead of directly modifying the original weight matrix W , LoRA introduces two smaller matrices, A and B , which together form a low-rank update to W .

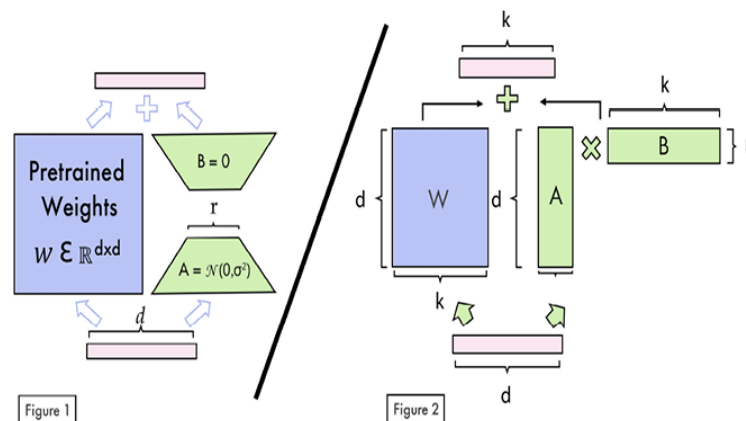


Figure 5.10 – LoRA adds the two trainable matrices A and B and keeps the pre-trained weights W frozen

Mathematically, this can be represented as:

$$W' = W + BA$$

Here, W is the original weight matrix, B and A are the LoRA matrices, and W' is the effective weight matrix used during inference.

The dimensions of matrices A and B are chosen such that their product has the same shape as W , but with a much lower rank. This rank, typically denoted as r , is a crucial hyperparameter in LoRA. During training, the original weights W remain frozen, while only A and B are updated. This approach significantly reduces the number of trainable parameters, leading to substantial memory savings and faster training times.

To implement LoRA effectively, we need to select the correct hyperparameters and target modules. LoRA comes with two hyperparameters:

- **Rank (r):** Determines the size of the LoRA matrices. A common starting point is $r = 8$, but values up to 256 have shown good results in some cases. Larger ranks may capture more diverse tasks but could lead to overfitting.
- **Alpha (α):** A scaling factor applied to the LoRA update. In practice, we update the frozen weights W by a factor of α/r . This is why a common heuristic is to set α to twice the value of r , effectively applying a scaling factor of 2 to the LoRA update. You can experiment with different ratios in case of overfitting or underfitting.

In addition, it is possible to add a drop-out layer to prevent overfitting. The dropout rate is usually set between 0 and 0.1 as an optional regularization factor, which slightly decreases training speed.

LoRA can be applied to various parts of the model architecture. Initially, LoRA was primarily focused on modifying the attention mechanism, specifically the **query (Q)** and **value (V)** matrices in transformer layers. However, experiments have demonstrated significant benefits in extending LoRA's application to other key components of the model. These additional target modules include:

- **Key (K)** matrices in attention layers
- Output projection layers (often denoted as O) in attention mechanisms
- Feed-forward or **Multi-Layer Perceptron (MLP)** blocks between attention layers
- Linear output layers

However, it's important to note that increasing the number of LoRA-adapted modules also increases the number of trainable parameters and, consequently, the memory requirements.

Using LoRA, it's possible to fine-tune a 7B parameter model on a single GPU with as little as 14-18 GB of VRAM, depending on the specific configuration. This is a dramatic reduction compared to full fine-tuning, which

would typically require multiple high-end GPUs. In terms of trainable parameters, LoRA drastically reduces the number compared to full fine-tuning. For example, even when targeting every module with a rank of 16, a Llama 3 8 B model only has 42 million trainable LoRA parameters out of 8 billion parameters, which is 0.5196% of the model's parameters.

In terms of quality, LoRA can also achieve comparable or sometimes better results than full-fine-tuning. Multiple sets of LoRA weights can be combined for different tasks or domains, allowing flexible deployment and task switching without retraining. Different projects are specialized in multiple-LoRA serving, such as LoRAX. It's also a feature supported by Hugging Face's **Text Generation Inference (TGI)** and **Nvidia Inference Microservices (NIM)**.

QLoRA

Introduced by Dettmers et al., QLoRA is a method for fine-tuning LLMs that addresses the challenges of high computational costs. By combining quantization techniques with LoRA, QLoRA allows developers to fine-tune models on relatively small, widely available GPUs.

The core of QLoRA's approach involves quantizing the base model parameters to a custom **4-bit NormalFloat (NF4)** data type, which significantly reduces memory usage. Like LoRA, instead of updating all model parameters during fine-tuning, QLoRA introduces small, trainable low-rank matrices (adapters) to specific layers of the model. Only these adapters are updated during training, while the original model weights remain unchanged. To further reduce memory usage, QLoRA employs double quantization, which quantizes the quantization constants themselves. Additionally, it uses paged optimizers to manage memory spikes during training by leveraging Nvidia's unified memory feature.

QLoRA provides significant memory savings compared to LoRA, reducing peak GPU memory usage by up to 75%. For example, for a 7B model, QLoRA reduces peak memory usage from 14 GB to 9.1 GB during initialization, a 35% reduction. During fine-tuning, the memory savings increase to 40%, from 15.6 GB for LoRA to 9.3 GB for QLoRA. However, this memory efficiency comes at the cost of increased training time, with QLoRA being about 30% slower than LoRA. In terms of model performance, QLoRA shows only minor differences compared to LoRA.

In summary, QLoRA is particularly beneficial when memory constraints are the primary concern, such as when working with very large models or on hardware with limited GPU memory. However, if training speed is crucial and sufficient memory is available, LoRA might be the preferred choice.

The decision between QLoRA and LoRA should be based on the specific requirements of the project, available hardware, and the need to balance memory usage, training speed, and model performance.

Training parameters

When fine-tuning LLMs, several hyperparameters guide the training process and significantly impact the model's convergence, generalization, and overall effectiveness.

Learning rate and scheduler

The learning rate is the most important hyperparameter. It controls how much the model's parameters are updated during training. It typically ranges from very small values like $1e-6$ to larger values like $1e-3$. A common starting point for transformer models is often around $1e-5$. If the learning rate is too low, training progresses slowly and may get stuck in suboptimal solutions. Conversely, if it's too high, training can become unstable or diverge, leading to poor performance. It's often beneficial to experiment with different learning rates to find the optimal value for your specific task and model.

The learning rate scheduler adjusts the learning rate throughout the training process. It typically starts with a higher learning rate to enable rapid initial progress, then gradually decreases it in later stages to fine-tune the model more precisely. The two most common types of schedulers are linear and cosine. A linear scheduler decreases the learning rate steadily over time, while a cosine scheduler follows a cosine curve, decreasing more slowly at first and then more rapidly toward the end of training. For example, you might start with a learning rate of $3e-4$ and decrease it to $1e-7$ over the course of training. The specific values and decay schedule depend on your model and dataset, but a common approach is to use a warmup period (e.g., 5% of total steps) where the learning rate increases from 0 to the initial value, followed by a decay period for the remaining 95% of steps. This approach helps stabilize early training and allows for more refined updates as the model converges. In general, linear and cosine schedulers provide the same level of performance.

Batch size

The batch size determines the number of samples processed before the model's weights are updated. Typical batch sizes for LLM fine-tuning range from 1 to 32, with common values being 1, 2, 4, 8, or 16. Larger batch sizes generally lead to more stable gradient estimates and can improve training speed, as they provide a better approximation of the true gradient of the entire dataset.

However, they also require more memory, which can be a limiting factor on GPUs with less VRAM. For instance, a batch size of 16 might work well on a high-end GPU with 24GB of memory, while a smaller GPU with 8 GB might only handle a batch size of 2 or 4.

To overcome memory constraints while still benefiting from larger batch sizes, a technique called gradient accumulation can be used. It works by performing multiple forward and backward passes with smaller mini-batches, accumulating the gradients over these steps before applying a single update to the model's parameters. This approach is particularly useful when working with large models or limited GPU memory. For example, if you want to achieve an effective batch size of 32 but your GPU can only handle 8 samples at a time, you can set the gradient accumulation steps to 4. This means you'll process 4 mini-batches of 8 samples each, accumulating the gradients, and then update the model as if you had processed all 32 samples at once.

The number of gradient accumulation steps typically ranges from 1 (no accumulation) to 8 or 16, depending on the desired effective batch size and available computational resources. When choosing the number of

steps, consider the trade-off between training speed and memory usage. More accumulation steps allow for larger effective batch sizes but increase the time required for each update. Here's a simple formula to determine the effective batch size:

$$\text{Effective Batch Size} = \text{Batch Size} \times \# \text{ GPUs} \times \text{Gradient Accumulation Steps}$$

For instance, if you're using 2 GPUs, each processing a batch of 4 samples, with 4 gradient accumulation steps, your effective batch size would be $4 * 2 * 4 = 32$ samples.

Maximum length and packing

The maximum sequence length determines the longest input the model can process. It's typically set between 512 and 4,096 tokens but can go up to 128,000 or more, depending on the task and available GPU memory. For example, a maximum length of 2,048 tokens is common for many language generation tasks, while RAG applications might use up to 8,192 tokens or more. When processing input data, sequences longer than this limit are truncated, meaning excess tokens are removed. Truncation can occur at the beginning (left truncation) or end (right truncation) of the sequence. For instance, with a maximum length of 1,024 tokens, a 1,500-token input would have 476 tokens removed. This parameter directly impacts batch size and memory usage; a batch size of 12 with a max length of 1,024 would contain 12,288 tokens ($12 * 1,024$), while the same batch size with a max length of 512 would only contain 6,144 tokens. It's important to balance this parameter with your GPU capabilities and the nature of your training data to optimize performance and resource utilization.

Packing maximizes the utilization of each training batch. Instead of assigning one sample per batch, packing combines multiple smaller samples into a single batch, effectively increasing the amount of data processed in each iteration. For example, if your maximum sequence length is 1,024 tokens, but many of your samples are only 200-300 tokens long, packing could allow you to fit 3-4 samples into each batch slot. This approach can significantly improve training efficiency, especially when dealing with datasets containing many short sequences. However, packing requires careful implementation to ensure that model attention doesn't cross between packed samples. This is typically achieved by using attention masks that prevent the model from attending to tokens from different samples within the same packed sequence.

Number of epochs

The number of epochs is another important parameter, representing the number of complete passes through the entire training dataset. For LLM fine-tuning, the typical range is 1 to 10 epochs, with many successful runs using 2 to 5 epochs. The optimal number depends on factors such as task complexity, dataset size, and model architecture. More epochs allow the model to refine its learning, potentially improving performance. However, there's a crucial trade-off: too few epochs may lead to underfitting, while too many can cause overfitting. For example, a large model fine-tuned on a small dataset might only need 1-3 epochs, while a smaller model fine-tuned on a larger dataset could benefit from 5-10 epochs. It is helpful to monitor validation performance during training and implement early stopping if the model's performance plateaus or degrades.

This approach helps determine the optimal number of epochs dynamically and prevents overfitting.

Optimizers

Optimizers adjust the model's parameters to minimize the loss function. For LLM fine-tuning, AdamW (Adaptive Moment Estimation with Weight Decay) is highly recommended, particularly its 8-bit version. AdamW 8-bit performs comparably to the 32-bit version while using less GPU memory (but it doesn't improve training speed). AdamW combines adaptive learning rates with weight decay regularization, often leading to better training stability and model performance.

For scenarios with severe memory constraints, AdaFactor presents an alternative designed for memory efficiency. It works well without explicit learning rate tuning, making it particularly useful in resource-constrained environments. However, it may not always match AdamW's performance in all cases. In situations involving extremely large models or limited GPU memory, paged versions of optimizers, such as paged AdamW 8-bit, can further reduce memory consumption by offloading to CPU RAM. If memory allows and maximum performance is the priority, the non-quantized `adamw_torch` optimizer may be the best choice.

Weight decay

Weight decay works by adding a penalty for large weights to the loss function, encouraging the model to learn simpler, more generalizable features. This helps the model avoid relying too heavily on any single input feature, which can improve its performance on unseen data. Typically, weight decay values range from 0.01 to 0.1, with 0.01 being a common starting point. For example, if you're using the AdamW optimizer, you might set the weight decay to 0.01.

While weight decay can be beneficial, setting it too high can impede learning by making it difficult for the model to capture important patterns in the data. Conversely, setting it too low may not provide sufficient regularization. The optimal weight decay value often depends on the specific model architecture and dataset, so it's generally a good practice to experiment with different values.

Gradient checkpointing

Gradient checkpointing is a technique that reduces memory consumption during training by storing only a subset of intermediate activations generated in the forward pass. In standard training procedures, all intermediate activations are retained in memory to facilitate gradient calculation during the backward pass. However, for very deep networks like LLMs, this approach can quickly become impractical due to hardware limitations, especially on GPUs with limited memory capacity.

Gradient checkpointing addresses this challenge by selectively saving activations at specific layers within the network. For layers where activations are not saved, they are recomputed during the backward pass as needed for gradient computation. This approach creates a trade-off between computation time and memory usage. While it significantly reduces memory requirements, it may increase overall computation time due to the need to recalculate some activations.

Other parameters and techniques exist but play a minor role compared to those previously discussed. In the next section, we will explore how to select and tune these parameters using a concrete example.

Fine-tuning in practice

Let's now fine-tune an open-source model on our custom dataset. In this section, we will show an example that implements LoRA and QLoRA for efficiency. Depending on the hardware you have available, you can select the technique that best corresponds to your configuration.

There are many efficient open-weight models we can leverage for task or domain-specific use cases. To select the most relevant LLM, we need to consider three main parameters:

- **License:** Some model licenses only allow non-commercial work, which is a problem if we want to fine-tune for a company. Custom licenses are common in this field, and can target companies with a certain number of users, for example.
- **Budget:** Models with smaller parameter sizes (<10 B) are a lot cheaper to fine-tune and deploy for inference than larger models. This is due to the fact that they can be run on cheaper GPUs and process more tokens per second.
- **Performance:** Evaluating the base model on general-purpose benchmarks or, even better, domain- or task-specific benchmarks relevant to the final use case, is crucial. This helps ensure that the model has the necessary capabilities to perform well on the intended tasks after fine-tuning.

In this chapter, we will choose Llama 3.1 8B, an open-weight model released by Meta. It has a permissive custom license ("Llama 3.1 Community License Agreement") that allows commercial use. With 8B parameters, it is small enough to fit on most GPUs while reaching a high level of performance compared to its competitors. We can verify this using the Open LLM Leaderboard, as well as other benchmarks detailed in the model card.

There are specialized tools and libraries to fine-tune models. In particular, we recommend the following:

- **TRL:** This is a library created and maintained by Hugging Face to train LLMs using SFT and preference alignment. It is a popular and reliable library that tends to be the most up-to-date in terms of algorithms. It works in single and multi-GPU settings with FSDP and DeepSpeed.
- **Axolotl:** Created by Wing Lian, this tool streamlines the fine-tuning of LLMs with reusable YAML configuration files. It is based on TRL but includes many additional features, such as automatically combining datasets stored in various formats. It also supports single- and multi-GPU settings with FSDP and DeepSpeed.
- **Unsloth:** Created by Daniel and Michael Han, Unsloth uses custom kernels to speed up training (2-5x) and reduce memory use (up to 80% less memory). It is based on TRL and provides many utilities, such as automatically converting models into the GGUF quantization format. At the time of writing, it is only available for single-GPU settings.

To maximize efficiency, we will perform fine-tuning using the Unsloth library. The following code is designed as part of our LLMops pipeline, but

can also be used as a stand-alone script. It can also be executed in different environments, like SageMaker, cloud GPUs (like Lambda Labs or RunPod), Google Colab, and many others. We tested it on different GPUs, like A40, A100, and L4.

To install the Unsloth library and its dependencies, we recommend directly installing from the GitHub repository of the book (<https://github.com/PacktPublishing/LLM-Engineering>) or Unsloth's repo (<https://github.com/unslothai/unsloth>). This approach is recommended because the installation steps are regularly updated to address potential conflicts with dependencies:

1. First, we want to access a gated model and (optionally) upload our fine-tuned model to Hugging Face (<https://huggingface.co/>). This requires being logged in to an account. If you don't have an account, you can create it and store your API key (**Settings | Access Tokens | Create new token**) in the .env file:

```
HF_TOKEN = YOUR_API_KEY
```

2. Make sure that your Comet ML API key is also in the .env file:

```
COMET_API_KEY = YOUR_API_KEY
```

3. Import all the necessary packages:

```
import os
import torch
from trl import SFTTrainer
from datasets import load_dataset, concatenate_datasets
from transformers import TrainingArguments, TextStreamer
from unsloth import Fast
```

4. Let's now load the model to fine-tune and its corresponding tokenizer. We use Unsloth's FastLanguageModel class with the `.from_pretrained()` method. In addition to the model name, we need to specify the max sequence length (2,048 in this example). Finally, the `load_in_4bit` argument indicates if we want to use **QLoRA (quantized pre-trained weights)** or LoRA.

We'll use LoRA in this example because of faster training and higher quality, but you can easily switch to QLoRA if you don't meet the VRAM requirements.

```
max_seq_length = 2048
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="meta-llama/Meta-Llama-3.1-8B",
    max_seq_length=max_seq_length,
    load_in_4bit=False,
)
```

5. Now that the model is loaded, we can define our LoRA configuration. Here, we use a rank of 32 that is large enough to imitate the writing style and copy the knowledge from our instruction samples. You can increase this value to 64 or 128 if your results are underwhelming. We also set an alpha of 32, without dropout and without bias, to speed up

training. Finally, we target every linear layer to maximize the quality of the fine-tuning process.

```
model = FastLanguageModel.get_peft_model(
    model,
    r=32,
    lora_alpha=32,
    lora_dropout=0,
    target_modules=["q_proj", "k_proj", "v_proj", "up_proj", "down_proj", "o_proj"]
)
```

6. Next, we need to prepare the data in the right format for fine-tuning. In this example, we don't have a lot of samples in the llmtwin dataset (3,000 samples). This is an issue because the model might not correctly learn the chat template. To address this, we will upsample it with a high-quality general-purpose dataset called FineTome. This is a filtered version of arcee-ai/The-Tome using the fineweb-edu-classifier. Instead of using the 100,000 samples of this dataset, we will specify we only want 10,000 in the train split. We concatenate these two datasets to create our final set.

```
dataset1 = load_dataset("mlabonne/llmtwin")
dataset2 = load_dataset("mlabonne/FineTome-Alpaca-100k", split="train[:10000]")
dataset = concatenate_datasets([dataset1, dataset2])
```

7. Now, we need to format this data using a chat template. Let's use the Alpaca template for convenience. This template doesn't require additional tokens, which makes it less error-prone (but can slightly impact performance compared to ChatML). Here, we map all the instructions and answers to the Alpaca template. We manually add the **end of sentence (EOS)** token at the end of each message to ensure that the model learns to output it. Without it, it will keep generating answers without ever stopping.

```
alpaca_template = """Below is an instruction that describes a task. Write a response
### Instruction:
{}
### Response:
{}"""
EOS_TOKEN = tokenizer.eos_token
dataset = dataset.map(format_samples, batched=True, remove_columns=dataset.column_names)
```

8. Once the dataset is ready, we can divide it into training (95%) and test (5%) sets for validation during training.

```
dataset = dataset.train_test_split(test_size=0.05)
```

9. The model is now ready to be trained. The SFTTrainer() class stores all the hyperparameters for our training. In addition, we provide the model, tokenizer, LoRA configuration, and datasets. Following the recommendations from the previous section, we set a learning rate of $3e-4$ with a linear scheduler and a maximum sequence length of 2048. We train this model for three epochs with a batch size of 2 and 8 gradient accumulation steps (for an effective batch size of 16). We also choose the adamw_8bit optimizer with a weight_decay of 0.01. Depending on the GPU we use, it will automatically use FP16 or BF16

for the activations. Finally, we report our training run to Comet ML for experiment tracking.

```

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    packing=True,
    args=TrainingArguments(
        learning_rate=3e-4,
        lr_scheduler_type="linear",
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,
        num_train_epochs=3,
        fp16=not is_bfloat16_supported(),
        bfloat16=is_bfloat16_supported(),
        logging_steps=1,
        optim="adamw_8bit",
        weight_decay=0.01,
        warmup_steps=10,
        output_dir="output",
        report_to="comet_ml",
        seed=0,
    ),
)
trainer.train()

```

Training this model on our concatenated dataset can take a few hours. For example, it takes 50 minutes on an A100 GPU.

10. Once it's done, we can test it with a quick example. The goal is not to properly evaluate the fine-tuned model, but to make sure that there are no obvious errors related to the tokenizer or chat template.

For fast inference, we can use `FastLanguageModel.for_inference()` from Unsloth. We directly format an instruction with the Alpaca format. Note that we provide an empty answer to append the assistant header (`### Response`): at the end of the user instruction. This forces the model to answer the instruction instead of completing it. We also use a text streamer to stream the generation instead of waiting for it to be complete before printing it.

```

FastLanguageModel.for_inference(model)
message = alpaca_prompt.format("Write a paragraph to introduce supervised fine-tun:
inputs = tokenizer([message], return_tensors='pt').to("cuda")
text_streamer = TextStreamer(tokenizer)
_ = model.generate(**inputs, streamer=text_streamer, max_new_tokens=256, use_cache=

```

11. Here is the answer provided by our model:

```

Supervised fine-tuning is a method used to enhance a language model by providing
responses with human expectations, thereby improving its accuracy and relevance.

```

This is correct and properly formatted with the Alpaca chat template.

12. Now that our model has been successfully fine-tuned, we can save it locally and/or push it to the Hugging Face Hub using the following functions.

```
model.save_pretrained_merged("model", tokenizer, save_method="merged_16bit")
model.push_to_hub_merged("mlabonne/TwinLlama-3.1-8B", tokenizer, save_method="me
```

Congratulations on fine-tuning a base model from scratch! During training, you can access Comet ML to monitor your training loss, validation loss, and many other metrics. You want to make sure that these metrics correspond to what is expected. *Figure 5.11* shows the training run corresponding to the previous code in Comet ML.

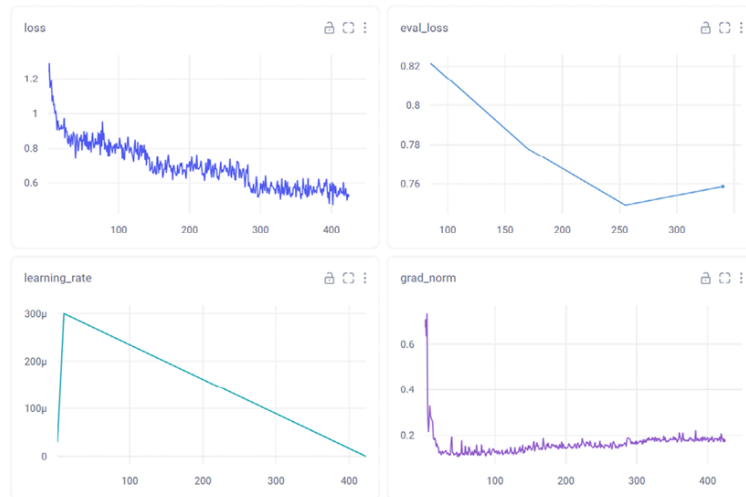


Figure 5.11 – Four monitored metrics during fine-tuning in Comet ML

In particular, three of these metrics are important to monitor:

- **Training loss:** It measures how well the model is performing on the task it's being trained for. The loss should continuously decrease on average, indicating improving performance. We expect a rapid decrease at the beginning of training, followed by a long plateau. Spikes and continuous increases in the loss value are signs that the training is failing. In this case, you might want to check the quality of your data, issues with the tokenizer, and tune parameters like learning rate and batch size. In *Figure 5.11* (loss), you can see three different phases corresponding to our three epochs.
- **Validation loss:** It measures the loss using the validation set instead of the training set; a well-fitted model typically shows both training and validation losses decreasing and eventually stabilizing, with a small gap between them. This gap should be minimal but is expected to exist as the model will always perform slightly better on the training data. If the training loss continues to decrease while the validation loss starts to increase, it's a sign of overfitting. Conversely, if both curves remain flat at a relatively high loss value, it indicates underfitting. There are no universal “recommended ranges” for loss values, as these depend on the specific problem and loss function used. However, you should look for convergence and stability in both curves. In *Figure 4.11* (eval_loss), we see a slight increase at step 340. This is still acceptable but might indicate that the model starts to overfit.

- **Gradient norm:** It represents the magnitude of the gradient vector during training. Large gradient norms can indicate training instability like overfitting, especially if accompanied by a divergence between training and validation losses. On the other hand, a stable or decreasing gradient norm generally means that the model is converging toward a local optimum. To mitigate issues associated with large gradient norms, gradient clipping can be employed. This technique involves setting a maximum threshold for the gradient norm, effectively limiting the size of parameter updates.

It is often interesting to try different learning rates and select the best model based on the minimal loss. Note that this is a proxy for real evaluations, which are covered in the next chapter.

Summary

This chapter covered essential aspects of LLM fine-tuning, both in theory and practice. We examined the instruction data pipeline and how to create high-quality datasets, from curation to augmentation. Each pipeline stage offers optimization opportunities, particularly in quality assessment, data generation, and enhancement. This flexible pipeline can be adapted to your use cases by selecting the most relevant stages and techniques.

We applied this framework to real-world data from *Chapter 3*, using an LLM to convert raw text into instruction-answer pairs. We then explored SFT techniques. This included an analysis of SFT's advantages and limitations, methods for storing and parsing instruction datasets with chat templates, and an overview of three primary SFT techniques: full fine-tuning, LoRA, and QLoRA. We compared these methods based on their impact on memory usage, training efficiency, and output quality. The chapter concluded with a practical demonstration that involved fine-tuning a Llama 3.1 8 B model on our custom instruction dataset. This example highlighted key steps and implementation details for successful fine-tuning.

In the next chapter, we will use preference alignment techniques to create a new version of TwinLlama-3.1-8B. We will generate a new dataset with chosen and rejected answers that will help us calibrate the type of answers we expect from our model. We will detail many applications that can benefit from this framework and how to implement it.

References

- Tahori, Gulrajani, Zhang, Dubois, et al.. "Alpaca: A Strong, Replicable Instruction-Following Model" [crfm.stanford.edu](https://crfm.stanford.edu/2023/03/13/alpaca.html), March 13, 2023, <https://crfm.stanford.edu/2023/03/13/alpaca.html>.
- Subhabrata Mukherjee et al.. "Orca: Progressive Learning from Complex Explanation Traces of GPT-4." arXiv preprint arXiv:2306.02707, June 2023.
- Wing Lian and Bleys Goodson and Eugene Pentland and Austin Cook and Chanvichet Vong and "Teknium". "Open-Orca/OpenOrca." *huggingface.co*, 2023, <https://huggingface.co/datasets/Open-Orca/OpenOrca>.
- Weihao Zeng et al.. "Automatic Instruction Evolving for Large Language Models." arXiv preprint arXiv:2406.00770, June 2024.

- Chunting Zhou et al.. “*LIMA: Less Is More for Alignment.*” arXiv preprint arXiv:2305.11206, May 2023
- 01. AI. “*Yi: Open Foundation Models by 01.AI.*” arXiv preprint arXiv:2403.04652, March 2024.
- Alex Birch. “*LLM finetuning memory requirements.*” [blog.scottlogic.com](https://blog.scottlogic.com/2023/11/24/llm-mem.html), November 24, 2023, <https://blog.scottlogic.com/2023/11/24/llm-mem.html>.
- Quentin Anthony et al.. “*Transformer Math 101.*” blog.eleuther.ai, April 18, 2023, <https://blog.eleuther.ai/transformer-math/>.
- Edward J. Hu et al.. “*LoRA: Low-Rank Adaptation of Large Language Models.*” arXiv preprint arXiv:2106.09685, June 2021.
- Tim Dettmers et al.. “*QLoRA: Efficient Finetuning of Quantized LLMs.*” arXiv preprint arXiv:2305.14314, May 2023.

Unlock this book's exclusive benefits**now**

This book comes with additional benefits designed to elevate your learning experience.

<https://www.packtpub.com/unlock/9781836200079>

Note: Have your purchase invoice ready before you begin.

Answers collapsed